PRÁCTICA 02

# Programación Avanzada en Python

Tipado, closures, dispatch, clases abiertas, mixins y traits

| | |
|---|---|
| **Tema** | Programación avanzada y patrones de diseño en Python |
| **Lenguaje** | Python 3.14 |
| **Dependencia** | multipledispatch |
| **Modalidad** | Individual |
| **Fecha** | Octubre de 2026 |
| **Entrega** | Código fuente y documento de soluciones |

*Documento de soluciones con explicación, código fuente y evidencia de ejecución.*

# Contenido


- 1. Presentación
- 2. Tipado estático y dinámico
- 3. Duck typing y protocolos estructurales
- 4. Closures y binding
- 5. Multiple dispatch
- 6. Clases abiertas
- 7. Mixins y traits
- 8. Conclusiones
- Anexo A. Código completo
- Anexo B. Ejecución en Visual Studio y VS Code

# 1. Presentación

Esta práctica aplica conceptos avanzados de Python mediante ejercicios independientes que se integran en un único programa ejecutable. Se trabajan anotaciones de tipo, protocolos estructurales, closures, multiple dispatch, clases abiertas, mixins y traits.

El código no contiene comentarios ni marcas de edición. Cada concepto se implementa con nombres descriptivos y se prueba con una salida controlada desde la función main.

| Elemento | Detalle |
|---|---|
| Archivo principal | practica_avanzada.py |
| Dependencia | multipledispatch>=1.0 |
| Ejecución | py practica_avanzada.py |
| Instalación | py -m pip install -r requirements.txt |
| Salida | consola |

# 2. Tipado estático y dinámico

Python es un lenguaje de tipado dinámico: los tipos se verifican en tiempo de ejecución. Las anotaciones de tipo no cambian el comportamiento del programa, pero documentan la interfaz de las funciones y permiten que editores y analizadores estáticos detecten errores antes de ejecutar.

| Anotación | Interpretación |
|---|---|
| list[int] | Lista de números enteros. |
| dict[str, float \| int] | Diccionario con claves de texto y valores numéricos. |
| -> dict[str, float \| int] | La función devuelve un diccionario con esa estructura. |

## 2.1 Función procesar_datos

**ENUNCIADO**

Crea una función llamada procesar_datos que acepte una lista de enteros y retorne un diccionario con la suma total, el promedio, el máximo y el mínimo. Usa anotaciones de tipo adecuadas.

**SOLUCIÓN**

La función valida que la lista no esté vacía, calcula los cuatro indicadores y devuelve un diccionario tipado. El promedio se obtiene dividiendo la suma entre la cantidad de elementos.

**CÓDIGO**

```
def procesar_datos(numeros: list[int]) -> dict[str, float | int]:
    if not numeros:
        raise ValueError("La lista no puede estar vacia.")
    suma = sum(numeros)
    return {
        "suma": suma,
        "promedio": suma / len(numeros),
        "maximo": max(numeros),
        "minimo": min(numeros),
    }
```

**SALIDA**

```
Suma: 150
Promedio: 30.0
Maximo: 50
Minimo: 10
```

# 3. Duck typing y protocolos estructurales

El duck typing se basa en el comportamiento: si un objeto implementa el método activar, puede utilizarse como dispositivo aunque no herede de una clase común. El protocolo Dispositivo describe esa interfaz para el analizador estático sin introducir una verificación explícita en tiempo de ejecución.

## 3.1 Clases Telefono, Alarma y Computadora

**ENUNCIADO**

Crea tres clases: Telefono, Alarma y Computadora. Cada una debe tener un método activar que imprima un mensaje distinto.

**SOLUCIÓN**

Cada clase implementa activar con un mensaje propio. No comparten una clase base, lo que permite demostrar que el contrato es el método y no la jerarquía.

**CÓDIGO**

```
class Telefono:
    def activar(self) -> None:
        print("Telefono activado: llamadas y mensajes disponibles.")

class Alarma:
    def activar(self) -> None:
        print("Alarma activada: sistema de seguridad en vigilancia.")

class Computadora:
    def activar(self) -> None:
        print("Computadora activada: sesion iniciada correctamente.")
```

**SALIDA**

```
Telefono activado: llamadas y mensajes disponibles.
Alarma activada: sistema de seguridad en vigilancia.
Computadora activada: sesion iniciada correctamente.
```

## 3.2 Función ejecutar_dispositivo

**ENUNCIADO**

Escribe una función ejecutar_dispositivo que use duck typing para llamar al método activar sin verificar tipos explícitos.

**SOLUCIÓN**

La función recibe un objeto que cumple el protocolo Dispositivo y llama a activar. No se utilizan isinstance ni comprobaciones de clase, por lo que cualquier objeto con ese método es aceptado.

**CÓDIGO**

```
class Dispositivo(Protocol):
    def activar(self) -> None:
        ...

def ejecutar_dispositivo(dispositivo: Dispositivo) -> None:
    dispositivo.activar()
```

**SALIDA**

```
Telefono activado: llamadas y mensajes disponibles.
Alarma activada: sistema de seguridad en vigilancia.
Computadora activada: sesion iniciada correctamente.
```

# 4. Closures y binding

Un closure es una función interna que conserva el acceso a las variables del ámbito donde fue creada. La palabra clave nonlocal permite modificar la variable capturada sin convertirla en global.

## 4.1 Closure crear_incrementador

**ENUNCIADO**

Escribe una función crear_incrementador(inicio) que retorne un closure que incremente una cuenta comenzando desde inicio.

**SOLUCIÓN**

La función externa define la variable cuenta y la función interna la modifica con nonlocal. Cada llamada a crear_incrementador produce un contador independiente.

**CÓDIGO**

```
def crear_incrementador(inicio: int):
    cuenta = inicio

    def incrementar(paso: int = 1) -> int:
        nonlocal cuenta
        cuenta += paso
        return cuenta

    return incrementar
```

**SALIDA**

```
Incremento 1: 11
Incremento 2: 16
Incremento 3: 18
```

## 4.2 Late binding y early binding

**ENUNCIADO**

Demuestra el problema de late binding y luego resuélvelo con early binding usando parámetros por defecto en una lista de funciones dentro de un bucle.

**SOLUCIÓN**

En el late binding, las lambdas capturan la misma variable indice y todas ven su último valor, por eso devuelven [2, 2, 2]. En el early binding, el valor se copia en un parámetro por defecto en cada iteración y se obtiene [0, 1, 2].

**CÓDIGO**

```
def demostrar_late_binding() -> tuple[list[int], list[int]]:
    funciones_tardias = [lambda: indice for indice in range(3)]
    funciones_tempranas = [lambda indice=indice: indice for indice in range(3)]
    resultados_tardios = [funcion() for funcion in funciones_tardias]
    resultados_tempranos = [funcion() for funcion in funciones_tempranas]
    return resultados_tardios, resultados_tempranos
```

**SALIDA**

```
Late binding: [2, 2, 2]
Early binding: [0, 1, 2]
```

> Regla práctica: si una función debe recordar el valor de una variable de un bucle, se pasa ese valor como argumento por defecto o se crea una fábrica de funciones.

# 5. Multiple dispatch

multipledispatch permite elegir una implementación según los tipos de todos los argumentos. Es una alternativa a las cadenas de isinstance y a la sobrecarga manual. La dependencia se instala con py -m pip install multipledispatch.

## 5.1 Función mezclar con multiple dispatch

#### ENUNCIADO

Instala y usa multipledispatch. Define una función mezclar que sume dos enteros, concatene dos listas y una dos strings con un guion.

#### SOLUCIÓN

Cada firma se registra con el decorador dispatch. El despachador selecciona la implementación según los tipos de los dos argumentos y devuelve el resultado correspondiente.

#### CÓDIGO

```
@dispatch(int, int)
def mezclar(a: int, b: int) -> int:
    return a + b

@dispatch(list, list)
def mezclar(a: list, b: list) -> list:
    return a + b

@dispatch(str, str)
def mezclar(a: str, b: str) -> str:
    return f"{a}-{b}"
```

#### SALIDA

```
Enteros: 11
Listas: [1, 2, 3, 4]
Strings: Python-Avanzado
```

| Firma | Comportamiento | Ejemplo |
|---|---|---|
| (int, int) | Suma aritmética | mezclar(4, 7) → 11 |
| (list, list) | Concatenación | mezclar([1,2], [3,4]) → [1,2,3,4] |
| (str, str) | Unión con guion | mezclar('Python','Avanzado') → 'Python-Avanzado' |

# 6. Clases abiertas

Python permite modificar una clase después de su definición. Esta característica se conoce como clase abierta o monkey patching. Es útil en prototipos y pruebas, pero debe documentarse y controlarse en producción.

## 6.1 y 6.2 Clase Usuario y método saludar dinámico

#### ENUNCIADO

Define una clase Usuario con un atributo nombre. Después, sin modificar la clase original, agrega dinámicamente un método saludar que imprima Hola, soy {nombre}.

#### SOLUCIÓN

La clase Usuario solo declara el constructor. La función saludar_usuario se define fuera y se adjunta a la clase con setattr. El método recibe self y accede al atributo nombre como cualquier método normal.

#### CÓDIGO

```
class Usuario:
    def __init__(self, nombre: str) -> None:
        self.nombre = nombre

def saludar_usuario(self: Usuario) -> None:
    print(f"Hola, soy {self.nombre}")
```

#### SALIDA

```
Hola, soy Ana
```

#### CÓDIGO DE VINCULACIÓN

```
setattr(Usuario, "saludar", saludar_usuario)

usuario = Usuario("Ana")
usuario.saludar()
```

La salida demuestra que el método agregado después de la definición de la clase se comporta igual que un método declarado dentro de ella.

# 7. Mixins y traits

Un mixin es una clase que aporta comportamiento reutilizable sin representar una entidad completa. Un trait, en este ejercicio, se simula con una clase abstracta que exige la implementación de medir.

## 7.1, 7.2 y 7.3 LoggerMixin, Sensor y SensorAvanzado

#### ENUNCIADO

Crea un LoggerMixin con un método log que imprima mensajes con el prefijo [LOG]. Define una clase base Sensor con un atributo tipo y una clase SensorAvanzado que herede de Sensor y LoggerMixin, y use el mixin para imprimir el tipo de sensor.

#### SOLUCIÓN

LoggerMixin aporta el método log. Sensor guarda el tipo. SensorAvanzado combina ambos: inicializa el sensor y utiliza log para registrar la creación. El orden de herencia coloca el mixin a la derecha de la clase base.

#### CÓDIGO

```
class LoggerMixin:
    def log(self, mensaje: str) -> None:
        print(f"[LOG] {mensaje}")

class Sensor:
    def __init__(self, tipo: str) -> None:
        self.tipo = tipo

class SensorAvanzado(Sensor, LoggerMixin):
    def __init__(self, tipo: str) -> None:
        super().__init__(tipo)
        self.log(f"Sensor de {self.tipo} inicializado")

    def leer(self) -> str:
        return f"Lectura del sensor de {self.tipo}"
```

#### SALIDA

```
[LOG] Sensor de temperatura inicializado
Lectura del sensor de temperatura
```

## 7.4 Trait MedibleTrait

#### ENUNCIADO

Simula un trait MedibleTrait que exija el método medir. Crea dos clases que implementen este trait con su propio comportamiento.

#### SOLUCIÓN

MedibleTrait hereda de ABC y declara medir como método abstracto. Las clases SensorTemperatura y SensorHumedad implementan medir con valores distintos. Si una clase no implementa el método, no puede instanciarse.

#### CÓDIGO

```
class MedibleTrait(ABC):
    @abstractmethod
    def medir(self) -> float:
        raise NotImplementedError

class SensorTemperatura(MedibleTrait):
    def medir(self) -> float:
        return 23.5

class SensorHumedad(MedibleTrait):
    def medir(self) -> float:
        return 61.0
```

**SALIDA**

```
Medición: 23.5
Medición: 61.0
```

| Concepto | Función | Ventaja |
|---|---|---|
| Mixin | Aporta comportamiento auxiliar. | Evita duplicar código entre clases. |
| Trait | Define un contrato que debe cumplirse. | Garantiza una interfaz mínima. |
| Herencia múltiple | Combina clases base. | Permite componer capacidades. |

En una aplicación real, el mixin se elige cuando se busca reutilizar una implementación concreta, como el registro de eventos, y el trait cuando se necesita garantizar un contrato mínimo entre clases que no comparten jerarquía. La decisión afecta la forma en que se documenta y se prueba cada componente.

> Criterio de diseño: usar composición y contratos pequeños reduce el acoplamiento y facilita sustituir una implementación sin modificar a los consumidores.

# 8. Conclusiones

La práctica demuestra que Python permite combinar tipado gradual, contratos estructurales y composición de comportamiento sin perder claridad. Las anotaciones de tipo mejoran la legibilidad y el análisis estático; los protocolos permiten trabajar por comportamiento; los closures conservan estado; el multiple dispatch elimina cadenas de condiciones; las clases abiertas facilitan la extensión; y los mixins y traits favorecen la reutilización. El código entregado está libre de comentarios y listo para ejecutarse en Visual Studio o VS Code.

# Anexo A. Código completo

Se incluye el archivo practica_avanzada.py completo, sin comentarios ni marcas de edición.

```
from abc import ABC, abstractmethod
from typing import Protocol

from multipledispatch import dispatch


def procesar_datos(numeros: list[int]) -> dict[str, float | int]:
    if not numeros:
        raise ValueError("La lista no puede estar vacia.")
    suma = sum(numeros)
    return {
        "suma": suma,
        "promedio": suma / len(numeros),
        "maximo": max(numeros),
        "minimo": min(numeros),
    }


class Dispositivo(Protocol):
    def activar(self) -> None:
        ...


class Telefono:
    def activar(self) -> None:
        print("Telefono activado: llamadas y mensajes disponibles.")


class Alarma:
    def activar(self) -> None:
        print("Alarma activada: sistema de seguridad en vigilancia.")


class Computadora:
    def activar(self) -> None:
        print("Computadora activada: sesion iniciada correctamente.")


def ejecutar_dispositivo(dispositivo: Dispositivo) -> None:
    dispositivo.activar()


def crear_incrementador(inicio: int):
    cuenta = inicio

    def incrementar(paso: int = 1) -> int:
        nonlocal cuenta
        cuenta += paso
        return cuenta

    return incrementar


def demostrar_late_binding() -> tuple[list[int], list[int]]:
    funciones_tardias = [lambda: indice for indice in range(3)]
    funciones_tempranas = [lambda indice=indice: indice for indice in range(3)]
    resultados_tardios = [funcion() for funcion in funciones_tardias]
    resultados_tempranos = [funcion() for funcion in funciones_tempranas]
    return resultados_tardios, resultados_tempranos


@dispatch(int, int)
def mezclar(a: int, b: int) -> int:
    return a + b


@dispatch(list, list)
def mezclar(a: list, b: list) -> list:
    return a + b


@dispatch(str, str)
def mezclar(a: str, b: str) -> str:
    return f"{a}-{b}"


class Usuario:
    def __init__(self, nombre: str) -> None:
        self.nombre = nombre
```

```
def saludar_usuario(self: Usuario) -> None:
    print(f"Hola, soy {self.nombre}")


setattr(Usuario, "saludar", saludar_usuario)


class LoggerMixin:
    def log(self, mensaje: str) -> None:
        print(f"[LOG] {mensaje}")


class Sensor:
    def __init__(self, tipo: str) -> None:
        self.tipo = tipo


class SensorAvanzado(Sensor, LoggerMixin):
    def __init__(self, tipo: str) -> None:
        super().__init__(tipo)
        self.log(f"Sensor de {self.tipo} inicializado")

    def leer(self) -> str:
        return f"Lectura del sensor de {self.tipo}"


class MedibleTrait(ABC):
    @abstractmethod
    def medir(self) -> float:
        raise NotImplementedError


class SensorTemperatura(MedibleTrait):
    def medir(self) -> float:
        return 23.5


class SensorHumedad(MedibleTrait):
    def medir(self) -> float:
        return 61.0


def mostrar_seccion(titulo: str) -> None:
    print("\n" + "=" * 68)
    print(titulo)
    print("=" * 68)


def main() -> None:
    mostrar_seccion("PARTE 1 - TIPADO ESTATICO Y DINAMICO")
    datos = procesar_datos([10, 20, 30, 40, 50])
    for clave, valor in datos.items():
        print(f"{clave.capitalize()}: {valor}")

    mostrar_seccion("PARTE 2 - DUCK TYPING Y PROTOCOLOS ESTRUCTURALES")
    for dispositivo in (Telefono(), Alarma(), Computadora()):
        ejecutar_dispositivo(dispositivo)

    mostrar_seccion("PARTE 3 - CLOSURES Y BINDING")
    incrementador = crear_incrementador(10)
    print(f"Incremento 1: {incrementador()}")
    print(f"Incremento 2: {incrementador(5)}")
    print(f"Incremento 3: {incrementador(2)}")

    tardios, tempranos = demostrar_late_binding()
    print(f"Late binding: {tardios}")
    print(f"Early binding: {tempranos}")

    mostrar_seccion("PARTE 4 - MULTIPLE DISPATCH")
    print(f"Enteros: {mezclar(4, 7)}")
    print(f"Listas: {mezclar([1, 2], [3, 4])}")
    print(f"Strings: {mezclar('Python', 'Avanzado')}")

    mostrar_seccion("PARTE 5 - CLASES ABIERTAS")
    usuario = Usuario("Ana")
    usuario.saludar()

    mostrar_seccion("PARTE 6 - MIXINS Y TRAITS")
    sensor = SensorAvanzado("temperatura")
    print(sensor.leer())

    medibles: list[MedibleTrait] = [SensorTemperatura(), SensorHumedad()]
    for medible in medibles:
        print(f"Medicion: {medible.medir()}")
```

```
if __name__ == "__main__":
    main()
```

# Anexo B. Ejecución en Visual Studio y VS Code

La práctica se ejecuta como un único archivo. Los pasos siguientes preparan el entorno y muestran el resultado en consola.

1. Abrir la carpeta 02_Programacion_Avanzada_Python en Visual Studio o VS Code.
2. Crear el entorno virtual con py -m venv .venv.
3. Activar el entorno con .\.venv\Scripts\Activate.ps1 en PowerShell.
4. Instalar la dependencia con py -m pip install -r requirements.txt.
5. Ejecutar el programa con py practica_avanzada.py.
6. En Visual Studio, seleccionar el intérprete y presionar F5; en VS Code, usar Run Python File.

| Elemento | Detalle |
|---|---|
| Archivo | practica_avanzada.py |
| Dependencia | multipledispatch |
| Ejecución | py practica_avanzada.py |
| Salida | consola |